

AgrotelliSystem

Sistema de Gestão de Produtos

Documentação de Entrega Final

Refatoração · Integração · CI/CD · Deploy Automatizado

Projeto	AgrotelliSystem — Gestão de Produtos
Versão	1.0.0
Tecnologia	Java 21 · Spring Boot · Thymeleaf · H2 · Gradle
CI/CD	GitHub Actions (ubuntu-latest)
Data	Abril / 2025

1. Arquitetura do Sistema

O AgrotelliSystem segue uma arquitetura em camadas (Layered Architecture), respeitando os princípios de Separation of Concerns e Single Responsibility. Todas as dependências fluem de fora para dentro: Controller → Service → Repository → Entity.

1.1 Diagrama de Camadas

Camada	Componentes	Responsabilidade
Presentation	ProductController, HomeController, GlobalExceptionHandler	Receber requisições HTTP, validar entrada, redirecionar resposta
Application	ProductService	Orquestrar regras de negócio, coordenar transações
Domain	Product, ProductDTO, BusinessException, ProductNotFoundException	Representar entidades e invariantes de domínio
Infrastructure	ProductRepository, ProductMapper	Persistência JPA, mapeamento DTO ↔ Entidade
UI / Templates	Thymeleaf templates, HTMX, Bootstrap 5, CSS custom	Renderização server-side, interações assíncronas parciais

1.2 Fluxo de Requisição

O fluxo completo de uma requisição segue a sequência abaixo:

1. Usuário acessa /products no browser
2. ProductController.list() é invocado pelo DispatcherServlet
3. ProductService.search() aplica filtros via ProductRepository
4. O repositório executa query JPQL com parâmetros opcionais
5. O resultado retorna como List<Product> ao controller
6. O controller adiciona atributos ao Model e retorna a view "products/list"
7. Thymeleaf renderiza o HTML e o browser exibe a tabela de produtos

1.3 Tecnologias Utilizadas

Tecnologia	Versão	Finalidade
Java	21 LTS	Linguagem principal (LTS)
Spring Boot	3.x	Framework web e IoC
Spring Data JPA	3.x	Abstração de persistência
H2 Database	2.x	Banco em memória para dev/test
Thymeleaf	3.x	Template engine server-side
HTMX	1.9.12	Requisições parciais sem SPA
Bootstrap	5.3.3	Framework CSS responsivo

Gradle	8.x	Build tool e gerência de deps
Lombok	1.18.x	Redução de boilerplate
Selenium	4.x	Testes pós-deploy E2E
JUnit 5	5.x	Testes unitários e integração

2. Refatoração e Qualidade de Código

O código já apresentava alta aderência a Clean Code e princípios SOLID. A análise identificou pontos de refinamento aplicados durante o processo de refatoração guiada por testes.

2.1 Princípios Aplicados

Single Responsibility Principle (SRP)

Cada classe possui uma única razão para mudar:

- ProductController — exclusivamente gerencia o ciclo de vida das requisições HTTP
- ProductService — contém toda a lógica de negócio (validação de unicidade, busca, CRUD)
- ProductMapper — responsável apenas pela conversão DTO ↔ Entity
- ProductRepository — abstração de acesso a dados via Spring Data JPA
- GlobalExceptionHandler — centraliza o tratamento de exceções evitando duplicação nos controllers

Imutabilidade e Separação Consulta/Modificação

O ProductService foi anotado com `@Transactional(readOnly = true)` no nível da classe, e apenas os métodos que modificam estado possuem `@Transactional` explícito. Isso garante:

- Consultas operam em modo read-only, melhorando performance e prevenindo dirty writes acidentais
- Modificações (create, update, delete) possuem transações completas com rollback automático
- A separação clara entre operações de leitura e escrita facilita a evolução para CQRS futuramente

Encapsulamento e Value Objects

Valores primitivos com semântica de negócio foram encapsulados no DTO com validações Bean Validation, evitando que strings inválidas cheguem à camada de serviço:

- `@NotBlank`, `@Size` — garantem strings válidas no nome e categoria
- `@DecimalMin`, `@DecimalMax` — restringem valores de preço ao domínio aceitável
- `@Min`, `@Max` — controlam limites de estoque

Guard Clauses e Fail-Fast

O método `findById()` do ProductService aplica guard clause logo no início:

```
if (id == null || id <= 0) { throw new BusinessException("ID inválido"); }
```

Isso elimina condicionais aninhadas e torna o fluxo principal mais legível.

Eliminação de Código Duplicado

O padrão de recarregar atributos do formulário em caso de erro (categories, formAction, formTitle) foi identificado como código repetitivo no ProductController. A refatoração extrai esse comportamento para métodos privados auxiliares, mantendo os métodos de handler concisos e focados.

2.2 Melhorias Implementadas

Área	Antes	Depois
Transações	Anotação manual em cada método	readOnly=true na classe + @Transactional nos mutadores
Validação ID	Sem verificação antes da query	Guard clause com BusinessException
Logging	Ausente em alguns fluxos	@Slf4j com log estruturado em todos os handlers
Naming	Variáveis genéricas (p, dto)	Nomes expressivos refletindo domínio
Exception hierarchy	Exceções genéricas	BusinessException + ProductNotFoundException especializadas

3. Workflows GitHub Actions

O pipeline de CI/CD foi estruturado em quatro arquivos YAML independentes, cada um com responsabilidade bem definida. Todos os workflows são executados em runners hospedados pelo GitHub (ubuntu-latest), eliminando a necessidade de infraestrutura dedicada para CI.

3.1 Visão Geral dos Workflows

Arquivo YAML	Triggers	Propósito
ci.yml	push, PR, manual	Build, testes unitários/integração, análise estática (Checkstyle)
security.yml	push, PR, schedule semanal	OWASP Dependency Check, CodeQL SAST, Gitleaks secret scan
post-deploy-selenium.yml	workflow_run, manual	Testes E2E Selenium headless validando integridade pós-deploy

3.2 Workflow CI (ci.yml) — Detalhamento

O workflow de Integração Contínua executa três jobs em paralelo após o build inicial:

Job: build

- Checkout do repositório com actions/checkout@v4
- Configuração do JDK 21 Temurin com cache do Gradle (actions/setup-java@v4)
- Build sem testes para validação rápida de compilação
- Upload do JAR como artefato com retenção de 7 dias
- Geração de resumo Markdown no \$GITHUB_STEP_SUMMARY

Job: test (depends on build)

- Execução de todos os testes com ./gradlew test
- Upload dos relatórios HTML e XML de testes com retenção de 14 dias
- Parsing dos XML para exibir contagem de testes no resumo do job

Job: static-analysis (depends on build, paralelo a test)

- Execução do Checkstyle via ./gradlew checkstyleMain checkstyleTest
- Upload do relatório HTML de análise estática
- Falha não bloqueia o pipeline (|| true) — apenas reporta

3.3 Workflow de Segurança (security.yml)

OWASP Dependency Check

Analisa todas as dependências do projeto contra a base NVD (National Vulnerability Database). O relatório HTML é armazenado como artefato por 30 dias para auditoria.

CodeQL SAST

Análise estática de segurança profunda utilizando o motor semântico do GitHub. Os resultados são publicados diretamente na aba Security → Code Scanning do repositório, permitindo triagem e gestão de vulnerabilidades inline no GitHub.

Gitleaks

Verifica o histórico completo de commits em busca de credenciais, tokens e segredos acidentalmente commitados. Executa com fetch-depth: 0 para garantir análise completa do histórico.

3.4 Workflow Selenium (post-deploy-selenium.yml)

Após cada deploy bem-sucedido, o workflow de testes pós-deploy é disparado automaticamente via workflow_run. Os testes validam o sistema em produção através de um browser Chrome headless:

- Verificação de carregamento da página de produtos (/products)
- Teste do formulário de criação de produto (/products/new)
- Validação do filtro de busca e categoria via HTMX
- Teste do modal de exclusão e confirmação
- Screenshots automáticos em caso de falha para diagnóstico

3.5 Gerenciamento de Segredos e Variáveis

Secret / Variável	Tipo	Uso
DEV_HOST / TEST_HOST / PROD_HOST	Secret	IP/hostname do servidor de destino
*_SSH_USER / *_SSH_KEY	Secret	Credenciais SSH para deploy via SCP
GITHUB_TOKEN	Builtin	Autenticação automática do GitHub Actions (CodeQL, Gitleaks)
DEV_APP_URL / TEST_APP_URL / PROD_APP_URL	Variable	URLs exibidas como environment URL no GitHub
SELENIUM_BASE_URL	Env	URL base usada pelos testes Selenium

4. Estratégias de Testes

4.1 Pirâmide de Testes Adotada

Nível	Ferramentas	O que valida
Unidade	JUnit 5, Mockito	ProductService, ProductMapper, validações de domínio
Integração	Spring Boot Test, H2	Fluxos controller → service → repository com banco em memória
E2E	Selenium 4, ChromeDriver	Fluxos completos no browser: CRUD de produtos, filtros, modal de exclusão

4.2 Testes Pós-Deploy com Selenium

Os testes Selenium são executados exclusivamente após um deploy bem-sucedido, garantindo que o sistema no ambiente alvo funcione corretamente com dados e configuração reais:

Cenários Cobertos

- Acesso à lista de produtos: verifica status HTTP 200 e presença da tabela #productsTable
- Criação de produto: preenche formulário, submete e confirma redirect com mensagem de sucesso
- Filtro por categoria: seleciona categoria e verifica que HTMX atualiza a tabela corretamente
- Exclusão de produto: abre modal de confirmação, confirma e valida remoção da linha
- Validação de formulário: submete formulário vazio e verifica mensagens de erro de validação

Configuração de Ambiente

```
# Variáveis de ambiente para Selenium
SELENIUM_BASE_URL=http://app.example.com
SELENIUM_HEADLESS=true
SELENIUM_TIMEOUT=30
```

4.3 Monitoramento e Logs de Workflow

Cada workflow implementa logs personalizados usando os comandos de workflow do GitHub Actions:

- `::notice` — informações relevantes exibidas no resumo do run
- `::warning` — alertas que não bloqueiam o pipeline
- `::error` — falhas críticas com contexto para diagnóstico
- `::group` / `::endgroup` — agrupamento de logs longos para melhor legibilidade
- `$GITHUB_STEP_SUMMARY` — tabelas Markdown com métricas de cada job

5. Manual de Execução

5.1 Executar a Aplicação Localmente

Pré-requisitos

- Java 21+ instalado (JAVA_HOME configurado)
- Git instalado

Passos

8. Clone o repositório:

```
git clone https://github.com/seu-usuario/agrotelli-system.git cd agrotelli-system
```

9. Execute a aplicação:

```
./gradlew bootRun
```

10. Acesse no browser:

```
http://localhost:8080
```

5.2 Executar os Testes Localmente

11. Todos os testes:

```
./gradlew test
```

12. Apenas testes unitários:

```
./gradlew test --tests "*UnitTest"
```

13. Apenas Selenium (requer Chrome instalado):

```
BASE_URL=http://localhost:8080 ./gradlew test --tests "*SeleniumTest"
```

14. Ver relatório HTML:

```
open build/reports/tests/test/index.html
```

Resultado da suíte completa de testes — 187 testes, 0 falhas, 100% de sucesso:

Test Summary

187
tests

0
failures

0
ignored

1m14.40s
duration

100%
successful

PackagesClasses

Package	Tests	Failures	Ignored	Duration	Success rate
com.crud.config	5	0	0	0.062s	100%
com.crud.controller	26	0	0	0.734s	100%
com.crud.exception	5	0	0	0.009s	100%
com.crud.fuzz	49	0	0	0.379s	100%
com.crud.integration	10	0	0	0.093s	100%
com.crud.model	4	0	0	0.001s	100%
com.crud.repository	15	0	0	0.149s	100%
com.crud.selenium	18	0	0	1m12.62s	100%
com.crud.service	19	0	0	0.334s	100%
com.crud.validation	36	0	0	0.021s	100%

Test Summary

187
tests

0
failures

0
ignored

1m14.40s
duration

100%
successful

PackagesClasses

Class	Tests	Failures	Ignored	Duration	Success rate
com.crud.config.DataInitializerTest	5	0	0	0.062s	100%
com.crud.controller.GlobalExceptionHandlerTest	4	0	0	0.427s	100%
com.crud.controller.HomeControllerTest	1	0	0	0.006s	100%
com.crud.controller.ProductControllerTest	21	0	0	0.301s	100%
com.crud.exception.ExceptionTest	5	0	0	0.009s	100%
com.crud.fuzz.ProductFuzzTest	49	0	0	0.379s	100%
com.crud.integration.ProductIntegrationTest	10	0	0	0.093s	100%
com.crud.model.ProductModelTest	4	0	0	0.001s	100%
com.crud.repository.ProductRepositoryTest	15	0	0	0.149s	100%
com.crud.selenium.ProductSeleniumTest	18	0	0	1m12.62s	100%
com.crud.service.ProductServiceTest	19	0	0	0.334s	100%
com.crud.validation.ProductDTOValidationTest	36	0	0	0.021s	100%

5.3 Executar Workflows GitHub Actions

Workflow CI — automático

Disparado automaticamente em todo push para main ou develop, e em todo Pull Request. Não requer ação manual.

Workflow Security — manual ou agendado

Executa automaticamente toda segunda-feira às 08:00 UTC. Para execução manual, acesse Actions → "Security — SAST & Dependency Scan" → "Run workflow".

5.4 Interpretar os Resultados dos Workflows

15. Acesse a aba Actions do repositório
16. Clique no workflow run desejado
17. Cada job exibe ☒ (sucesso) ou ✖ (falha) com tempo de execução
18. Clique em um job para ver os logs detalhados com grupos expansíveis
19. No final de cada run, a aba "Summary" exibe as tabelas Markdown geradas
20. Artefatos (JARs, relatórios, screenshots) ficam disponíveis na seção Artifacts

5.5 Badges de Status

Adicione as badges ao README.md do repositório para visibilidade do estado do pipeline:

```
! [CI] (https://github.com/seu-usuario/agrotelli-system/actions/workflows/ci.yml/badge.svg)
! [CD] (https://github.com/seu-usuario/agrotelli-system/actions/workflows/cd.yml/badge.svg)
! [Security] (https://github.com/seu-usuario/agrotelli-system/actions/workflows/security.yml/badge.svg)
```

6CD. Checklist de Entrega

✓/✗	Requisito	Evidência
✓	SRP aplicado — classes coesas e focadas	ProductController, Service, Mapper, Repository
✓	Eliminação de código duplicado	Métodos auxiliares no controller
✓	Guard clauses e fail-fast	findById() com validação de ID
✓	Imutabilidade: readOnly=true + @Transactional nos mutadores	ProductService
✓	Value objects com Bean Validation	ProductDTO com annotations
✓	Exception hierarchy especializada	BusinessException, ProductNotFoundException
✓	Logging estruturado com @Slf4j	ProductService + GlobalExceptionHandler
✓	Workflow CI com build Gradle	.github/workflows/ci.yml
✓	Workflow CI com testes automatizados	ci.yml — job: test
✓	Análise estática de código (Checkstyle)	ci.yml — job: static-analysis
✓	Deploy automático multi-ambiente (dev/test/prod)	.github/workflows/cd.yml
✓	SAST com CodeQL	security.yml — job: codeql-sast
✓	DAST / Dependency scan (OWASP)	security.yml — job: dependency-scan
✓	Secret scan (Gitleaks)	security.yml — job: secret-scan
✓	Testes pós-deploy com Selenium	post-deploy-selenium.yml
✓	Proteção de ambientes com approvals	Environments: dev/test/prod
✓	Logs personalizados (::notice::error::group)	Todos os workflows
✓	Resumos Markdown em \$GITHUB_STEP_SUMMARY	Todos os jobs de todos os workflows
✓	Upload e gestão de artefatos	actions/upload-artifact@v4 em todos os workflows
✓	Badges de status no README	Seção 5.5 deste documento
✓	Documentação completa da entrega	Este documento

✓ Sistema aprovado para entrega

Todos os requisitos das três etapas foram implementados. O código está refatorado segundo princípios SOLID e Clean Code. Os quatro workflows de CI/CD estão configurados com build, testes, segurança, deploy progressivo e testes pós-deploy Selenium.

